

Informe de auditoria — LavadoCarros

Proyecto: LavadoCarros

Version: 1.0

Fecha: 15/09/2026

Elaborado por: Auditoria tecnica

Fecha: 2026-09-15 · **Version:** 1.0

Ruta: /home/server-dt/Documentos/lavado_de_carros

Resumen ejecutivo

Sistema de reservas de lavado de vehiculos a domicilio. **Es el proyecto mejor construido de los auditados hasta ahora.** La arquitectura esta bien separada en capas, la autentificacion y la autorizacion estan resueltas con cuidado, hay rate limiting razonado, la subida de archivos esta acotada y **el codigo explica sus propias decisiones** en comentarios que resultaron exactos al contrastarlos.

Tiene datos de un piloto funcional completo: 18 reservas que recorrieron todo el ciclo, con asignacion de lavador, evidencia, pago y calificacion.

Los problemas no estan en el codigo sino alrededor:

1. **PostgreSQL esta expuesto a la red local** (puerto 5437), sin que firewalld lo cubra.
2. **No hay ningun respaldo**, y aqui se perderian tanto la base como las evidencias fotograficas y los comprobantes de pago.
3. **El webhook de pagos no valida firma.** Hoy es inocuo porque la pasarela no esta conectada; sera grave el dia que se conecte.

Estado general

● **OPERATIVO CON OBSERVACIONES**

Arquitectura

Tres contenedores (React+nginx / Express+Prisma / PostgreSQL 15.18), sin cache ni colas, con integraciones de SMTP y Twilio. Separacion rutas → controladores → servicios → Prisma. Adecuada.

Codigo

Buena calidad. Destacan tres decisiones correctas y poco frecuentes:

- La autentificacion **revalida contra la base en cada peticion**, de modo que suspender una cuenta corta el acceso al instante en vez de esperar 24 h.
- El rate limiting usa **id de usuario** como clave, no IP, con el razonamiento documentado sobre el falseo de `X-Forwarded-For`.
- La verificacion de pertenencia esta presente: un cliente no puede pagar la reserva de otro.

Docker

Correcto en lo esencial: `depends_on` con `service_healthy`, volúmenes declarados, política de reinicio. Falta healthcheck en backend y frontend, y los tres puertos se publican en todas las interfaces.

Base de datos

PostgreSQL 15.18, 15 entidades, integridad referencial coherente (`RESTRICT` en lo identificatorio, `SET NULL` en lo circunstancial). Sin procedimientos ni triggers. `Usuario.rol` es texto libre en vez de enum.

Seguridad

3 hallazgos altos, 4 medios, 2 bajos, 1 informativo. **Ningun critico.** Lo fundamental —contrasenas con bcrypt, secreto JWT fuerte, autorizacion por rol, validacion de subidas, sin superficie de SQL Injection— esta bien resuelto.

Documentacion

Existia documentacion previa abundante (8 archivos). Se conserva completa y queda clasificada: 3 vigentes, 2 desactualizados, 1 incompleto, 1 duplicado, 1 no verificable. **La API ya publicaba Swagger**, que es la mejor documentacion tecnica que tenia el proyecto.

Backup

● **Inexistente**, y con tres elementos irrecuperables: base, archivos subidos y `.env`.

Operacion

Sin monitoreo ni healthcheck de aplicacion. Logs sin persistencia.

Mobile

- **N/A.**

Riesgos

Riesgo	Probabilidad	Impacto	Exposicion
Perdida total de datos del piloto	media	alto	alta — sin respaldo
Acceso directo a PostgreSQL desde la LAN	media	alto	alta
Fraude via webhook al conectar la pasarela	alta si se conecta sin corregir	alto	alta
Perdida de evidencias y comprobantes	media	alto	alta — son respaldo documental del servicio y del cobro
Crecimiento sin limite de uploads_data	media	medio	media
Perdida del codigo	baja	medio	media — sin remoto git

Inconsistencias

INC-01 — Documentacion previa anterior al codigo

`DOCUMENTACION_TECNICA.md`, `ANALISIS.md` y `MODELO_BASE_DATOS.md` son del 2026-08-28; el codigo siguio cambiando hasta el 2026-08-31 (rate limiting y flujo de comprobantes). **Revisar:** los tres documentos. **Impacto:** medio, quien los lea creera que no hay rate limiting.

INC-02 — Version del producto sin unificar

`backend` declara `1.0.0`, `frontend` declara `0.1.0`, y no hay etiquetas en git. **Impacto:** bajo, pero impide referirse a "la version X" en soporte.

INC-03 — `idea.md` duplica `IDEA_NEGOCIO.md`

Dos documentos de negocio solapados. **Impacto:** bajo.

INC-04 — Los roles validos viven en un comentario

El schema declara `rol String` y los cuatro valores validos estan solo en un comentario. Codigo y base no coinciden en el nivel de garantia.

Impacto: medio.

Documentacion faltante (antes de esta auditoria)

Faltaban: manual de usuario, manual de administrador, manual de soporte, documentacion de operacion, de continuidad, plan de pruebas y gestion de versiones. Todo queda creado en `docs/`.

Recomendaciones

ID	Hallazgo	Categoria	Severidad	Recomendacion
CONT-01	Sin respaldo de base, archivos ni secretos	Continuidad	● CRITICO	Respaldo diario de los tres. Script en docs/12_CONTINUIDAD
SEC-01	PostgreSQL alcanzable desde la LAN	Seguridad	● ALTO	Publicar como 127.0.0.1:5437:5432
SEC-02	Webhook de pagos sin validacion de firma	Seguridad	● ALTO	Resolver ANTES de conectar la pasarela. Validar HMAC del proveedor
SEC-03	API y Swagger expuestos sin autenticacion	Seguridad	● ALTO	API en 127.0.0.1 tras nginx; Swagger fuera de produccion
SEC-04	CORS refleja cualquier origen con credenciales	Seguridad	● MEDIO	Lista blanca de origenes
SEC-05	Sin cabeceras de seguridad	Seguridad	● MEDIO	Agregar helmet
SEC-07	rol como texto libre sin restriccion	Seguridad	● MEDIO	Convertir a enum de Prisma
OPS-01	Backend y frontend sin healthcheck	Operacion	● MEDIO	Usar GET /api/health , que ya existe
OPS-02	uploads_data crece sin politica de purga	Operacion	● MEDIO	Definir retencion
SEC-06	X-Powered-By expone el framework	Seguridad	● BAJO	Cubierto por helmet
SEC-08	Limite de cuerpo JSON de 20 MB	Seguridad	● BAJO	Bajar a 1-2 MB
INC-01	Documentacion previa desactualizada	Documentacion	● MEDIO	Actualizar o marcar como historica
GES-01	Repositorio sin remoto	Gestion	● MEDIO	Publicar en el git de la empresa
PEND-01	Matriz de permisos por rol sin verificar	Pruebas	● MEDIO	Completar la matriz de docs/13_PRUEBAS
NEG-01	Vigencia del piloto sin definir	Negocio	● MEDIO	Definir si pasa a produccion

Prioridad recomendada

1. **CONT-01** — respaldo. Hay datos de negocio reales que hoy se perderian.
2. **SEC-01** — cerrar PostgreSQL. Es un cambio de una linea.
3. **SEC-02** — webhook. **Bloqueante para conectar pagos**, no urgente mientras no se conecte.
4. **SEC-03** — exposicion de la API.
5. OPS-01, SEC-04, SEC-05, SEC-07.

6. El resto.

Ningun cambio fue aplicado durante esta auditoria.

Anexo — Inconsistencia de propiedad de archivos (detectada al cerrar)

Al intentar registrar la documentacion en git aparecio un problema de permisos. Diagnostico:

Dentro del proyecto conviven archivos de dos duenos distintos. Son de `root` , entre otros:

```
.env
.git/COMMIT_EDITMSG
.git/objects/ (varios subdirectorios)
backend/prisma/seed-demo.js
backend/prisma/demo-imagenes.js
backend/src/utls/credenciales.js
backend/src/utls/firmaArchivos.js
```

El resto pertenece a `server-dt` , que es el dueno del directorio y el usuario con el que se opera normalmente.

Causa: en algun momento se trabajo en el proyecto como `root` (docker, edicion directa o git), y esos archivos quedaron con ese dueno.

Impacto:

- `server-dt` **no puede hacer commit** en el repositorio: git falla con "*permisos insuficientes para agregar un objeto*".
- `.env` , que contiene los secretos, pertenece a `root`. El usuario habitual no puede leerlo ni rotarlo sin elevar privilegios.
- Un despliegue o actualizacion ejecutado como `server-dt` fallaria de forma poco evidente.

Recomendacion (no aplicada): unificar la propiedad del proyecto.

```
sudo chown -R server-dt:server-dt /home/server-dt/Documentos/lavado_de_carros
sudo chmod 600 /home/server-dt/Documentos/lavado_de_carros/.env
```

Y en adelante, no operar el proyecto como `root`. Esta correccion **no se aplico en esta auditoria** porque cambia permisos del sistema de archivos y excede el alcance de "analizar y documentar".

Severidad: 🟡 MEDIO (OPS-03).